

LOONGX: Rethinking Random Graphs for AI Training Clusters

Heng Xu[♣], Yiming Lei[♡], Jialong Sun[♣], Ying Zhou[♣], Yiting Xia[♡], Jialong Li[♣]
♣Shenzhen University of Advanced Technology, ♡Max Planck Institute for Informatics

Abstract

Multi-tenant AI training workloads are dominated by communication within each job’s assigned nodes, while still requiring bounded network diameter as larger jobs span broader regions of the cluster. Existing network topologies struggle to simultaneously provide dense local communication and low diameter under this workload structure. We present LOONGX, a locality-biased small-world topology for multi-tenant AI training clusters. LOONGX concentrates connectivity within structured local neighborhoods to support dense intra-job communication, while allocating only a small fraction of links to random long-range connections. By leveraging the small-world effect, LOONGX achieves expander-like diameter while preserving dense local connectivity. Large-scale simulations with five representative AI training models show that LOONGX reduces iteration time by up to 39.3% in isolated settings and up to 55.7% under multi-tenant training compared to widely used baselines. A testbed implementation further confirms that LOONGX’s performance trends are consistent with simulation results.

1 Introduction

AI training at scale places substantial pressure on the network [12, 17, 26, 32, 45]. Modern training jobs involve frequent, tightly synchronized communication, making performance highly sensitive to delays along shared network paths. In this paper, we focus on multi-tenant AI clusters, where many concurrent training jobs with heterogeneous sizes and communication demands coexist and evolve over time. As a result, synchronized traffic from multiple jobs increasingly overlaps on shared network resources, amplifying the impact of path length and structural bottlenecks. This creates a fundamental tension: *training traffic is largely confined within job regions, yet the network must still provide short global paths when jobs scale or fragment.*

Clos fabrics remain the default in cloud datacenters [3] and many production AI clusters [17, 35, 44]. Their multi-stage hierarchy offers high bisection bandwidth and operational simplicity, but synchronized traffic often traverses multiple

aggregation layers, creating shared bottlenecks under concurrent communication.

Expander-inspired topologies [51, 53, 59], constructed via uniformly random connectivity¹, achieve near-optimal diameter and short average path length, making them attractive from a graph-theoretic perspective. Recent production experience with AWS shows that the cabling challenges of random graphs can be addressed in production-scale deployments [9]. However, uniform randomness is poorly aligned with AI training workloads. The limitation of expander-style designs is not their diameter, but the uniform allocation of connectivity that ignores the strong spatial skew in training workloads. Training jobs are typically placed on nearby nodes, with most traffic confined within a localized region, yet expanders distribute links evenly across the entire cluster. As a result, local communication often traverses multi-hop paths spanning distant regions, increasing synchronization latency and contention. This weak locality renders expander topologies inefficient for AI training despite their favorable global properties.

Other structured topologies exhibit different limitations. Torus-based designs preserve local adjacency [26], but their diameter grows quickly with scale, pushing medium and large training jobs onto long paths. Dragonfly reduces diameter by organizing nodes into groups with dense internal connectivity [27, 50], but once jobs span multiple groups, synchronization performance degrades due to reliance on cross-group communication and becomes highly sensitive to placement. More recent structured designs, such as ZCube [60], introduce additional structural layers, incurring higher cost and complexity, and are primarily designed for single large job training rather than multi-tenant settings. These constraints limit the effectiveness of existing designs.

Taken together, these limitations reveal a common set of structural requirements for multi-tenant AI clusters. Communication is largely confined within the nodes assigned to each job, calling for strong local connectivity. At the same time, larger jobs and fragmented placements inevitably span

¹We use Expander graph and random graph interchangeably in this paper.

broader portions of the cluster, making bounded network diameter necessary to prevent excessive hop counts. While Torus-based designs provide dense local connectivity, their diameter grows quickly with scale. Furthermore, the network structure should provide consistent communication properties across different regions, avoiding the placement sensitivity observed in group-based designs like Dragonfly, where performance degrades once jobs cross structural boundaries. An effective topology must therefore satisfy the combined requirements of local connectivity, bounded diameter, and structural consistency to support AI training.

To meet the above requirements, we design LOONGX, a topology that *preserves the near-optimal diameter of random graphs while introducing dense local neighborhoods* aligned with AI communication structures. It allocates most connectivity among nearby nodes so that the majority of traffic follows short local paths, and augments this structure with a small number of randomized long-range links. This design exploits the small-world effect, where adding only a few random shortcuts to an otherwise structured network collapses the diameter. Unlike expander graphs that distribute connectivity uniformly across the network, LOONGX limits randomness to a small fraction of links, retaining around 20% randomized long-range connections that suffice to match the diameter and average path length of fully random expanders. In addition, LOONGX maintains a uniform construction across the cluster, ensuring consistent communication properties and avoiding the structural sensitivity observed in group-based designs.

We evaluate LOONGX using five representative large AI training models under both isolated and multi-tenant settings. In isolated training, LOONGX reduces iteration time by up to 39.3% compared to Dragonfly, and by 10.6%–16.6% compared to Clos-based designs. Under multi-tenant training, LOONGX reduces iteration time by up to 55.7% compared to Dragonfly, 20.7–25.9% compared to Clos-based designs, and up to 13% compared to Expander. We further validate LOONGX through a testbed implementation, observing performance trends consistent with simulation.

This paper makes the following contributions.

- We characterize the mismatch between AI training communication patterns and existing network topologies, and identify the structural limitations of widely used designs, including Clos, Rail-optimized, Torus, Dragonfly, Expander, and ZCube, in multi-tenant environments.
- We introduce LOONGX, a new topology that combines dense local connectivity with a small fraction of random long-range links. By leveraging the small-world effect, LOONGX achieves near-optimal diameter while preserving locality aligned with multi-tenant patterns.
- Through extensive large-scale simulations on representative AI training workloads, we show that LOONGX reduces iteration time under both isolated and multi-tenant settings, and further validate these results through a testbed implementation.

[This work does not raise any ethical issues.]

2 Background and Motivation

In this section, we describe AI workload characteristics (§2.1), the resulting topology requirements for AI training (§2.2), the limitations of existing topologies (§2.3), and the key observation that motivates LOONGX (§2.4).

2.1 Characteristics of AI Workloads

Multi-tenant variability. Multi-tenant AI clusters routinely serve many concurrent training jobs that differ in scale, duration, and communication demand. Small and medium jobs issue modest but frequent exchanges, while larger jobs involve more workers and generate substantial cross-rack traffic. The resulting aggregate load varies over time as jobs arrive, complete, or resize. Prior studies of Meta [17] and Alibaba [58] production traces reveal a wide spread of training job sizes, where many modest-scale jobs coexist with a smaller number of large ones. A topology must therefore support heterogeneous and time-varying demand rather than target a single workload profile.

Parallelism heterogeneity. Within each job, different parallelization strategies induce distinct communication patterns that stress the network in different ways. Data parallelism (DP) [29, 47] relies on synchronized collective operations across workers. Pipeline parallelism (PP) [25, 39] generates sequential stage-to-stage transfers that favor short paths with low latency variance. Tensor parallelism (TP) [49] partitions layers across workers and produces fine-grained exchanges that are sensitive to load imbalance. Expert parallelism (EP) [15, 33, 46, 61] creates concentrated all-to-all traffic with skewed but iteration-consistent destinations. While these patterns differ in structure and granularity, they recur across iterations and collectively exercise path length, spatial locality, and load balance in the underlying topology.

Synchronization sensitivity. Many large-scale training jobs progress in synchronized iterations, where workers exchange collective communication and must complete communication before computation resumes. Under such execution models, job progress is directly affected by hop count along communication paths. As a result, short paths and sufficient local connectivity are particularly important for reducing iteration time, especially for latency-critical or fine-grained exchanges.

2.2 Topology Requirements for AI Training

AI training workloads impose specific structural requirements on network topologies, which we summarize as the key properties most directly affecting training performance.

Path length. AI training progresses in synchronized iterations, where communication latency directly accumulates into iteration time. Short path lengths reduce synchronization delay and limit the amplification of queueing effects during collective operations.

Local connectivity. AI training traffic is dominated by communication among workers of the same job, which are typi-

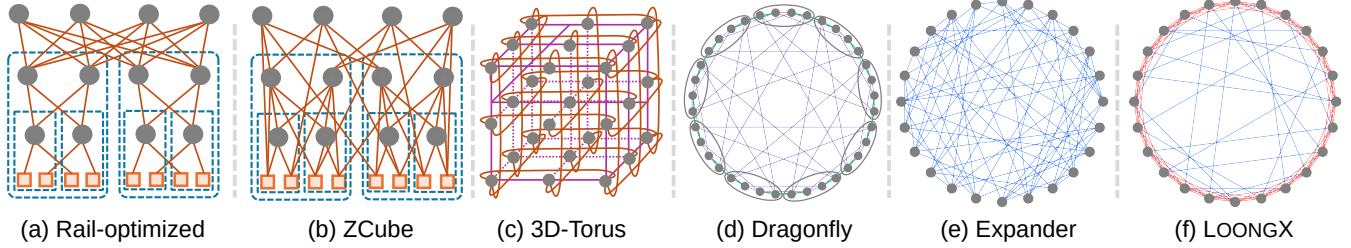


Figure 1: Topologies in AI clusters: (a) Rail-optimized, (b) ZCube, (c) 3D-Torus, (d) Dragonfly, (e) Expander, and (f) LOONGX.

Table 1: Comparison of representative topology families.

Topology	Path length	Local connectivity	Long-range connectivity	Structural consistency	Cost
Clos	**	**	**	**	**
Rail-optimized	**	***	**	**	*
ZCube	***	***	**	**	*
Torus	*	***	*	***	**
Dragonfly	***	***	**	*	**
Expander	***	*	***	***	**
LoongX	***	***	**	***	**

cally placed close to each other within the cluster. Strong local connectivity supports dense collectives and fine-grained exchanges while minimizing interference with unrelated traffic on distant links.

Long-range connectivity. While communication is often localized, large jobs and certain parallelization strategies still require efficient coordination across distant workers. Adequate long-range connectivity provides shortcuts that limit diameter growth and prevent global communication phases from becoming bottlenecks.

Structural consistency. Multi-tenant AI clusters place jobs dynamically across the network, and training performance should not depend sensitively on where workers are assigned. Structural consistency ensures that nodes experience similar path lengths and connectivity characteristics, avoiding systematically worse communication conditions.

Cost. AI training clusters must scale under practical hardware and wiring constraints, making cost an unavoidable consideration in topology design.

2.3 Why Existing Topologies Fall Short for AI Workloads

We examine why existing data center topologies fall short for AI training workloads. As illustrated in Fig. 1, current AI clusters mainly adopt two classes of designs: hierarchical topologies, such as Clos, Rail-optimized designs, and ZCube; and flat topologies, including Torus, Dragonfly, and Expander. Table 1 compares these representative topology families along key properties. The comparison shows that most designs optimize one or two properties at the expense of others. Consequently, existing topologies cannot simultaneously provide short paths, strong locality, and structural consistency for multi-tenant AI training under practical cost constraints.

Clos. Clos designs [3] are widely deployed in production data

centers due to their symmetric bisection bandwidth and simple wiring; they perform well for cloud workloads dominated by independent, throughput-driven flows. However, AI training relies on synchronized, iteration-bound collectives such as all-reduce and all-gather. In large Clos fabrics, these collectives traverse multiple stages over long multi-hop paths, increasing latency and exposing communication to repeated contention along the path. As a result, iteration time becomes highly sensitive to path length and contention.

Rail-optimized. Rail-optimized designs [1, 55] shorten paths for workers whose indices align under specific parallelization strategies. While this improves structured collective patterns, the underlying fabric remains Clos-like and provides limited rail-aligned path diversity. This becomes a bottleneck for all-to-all communication, where many distinct routes are needed and the preferred rails can quickly saturate. The benefits also diminish when placement deviates from idealized alignment or when concurrent jobs introduce unrelated traffic. As a result, Rail-optimized designs improve specific cases, but expose the same hierarchy-induced contention and placement sensitivity as the base Clos topology.

ZCube. Structured topologies such as ZCube [60] achieve low diameter through symmetric, recursive construction with relatively uniform hop counts. The connectivity pattern is tightly coupled to the node count and per-node port budget, requiring multi-port NICs and increasing hardware cost. Ports are assigned to different layers in a fixed manner, which limits flexibility in rebalancing local and long-range connectivity. ZCube is designed and evaluated primarily for single training jobs, and its behavior under concurrent multi-tenant workloads has not been explored. Under dynamic placement, communication performance can therefore depend on how jobs align with the predefined structure.

Torus. Torus topologies are used in accelerator-centric systems, including Google TPU deployments [26, 63], as they provide dense local connectivity. However, their diameter grows with system scale, resulting in long multi-hop paths for distant communication and the lack of efficient shortcuts. This makes Torus particularly inefficient for all-to-all communication and amplifies synchronization delays as job sizes grow or placements fragment.

Dragonfly. Dragonfly [27, 50] reduces diameter by organizing nodes into fixed groups with full-mesh internal connectivity, which supports efficient communication within a

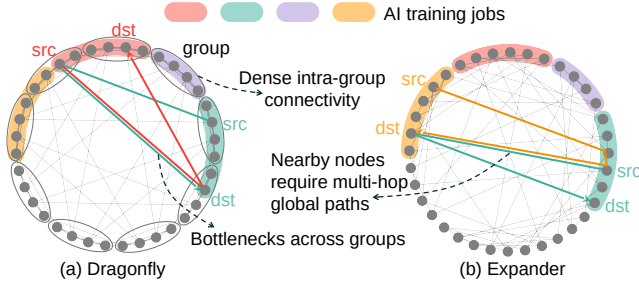


Figure 2: Limitations of (a) Dragonfly and (b) Expander.

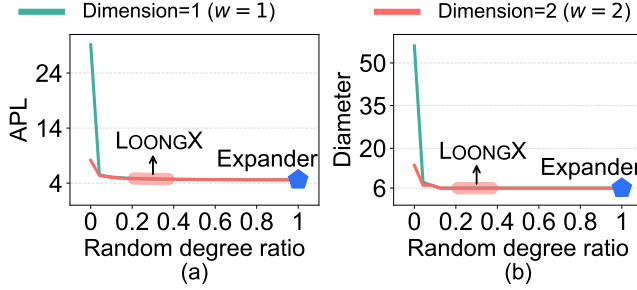


Figure 3: (a) APL and (b) diameter under varying random degree ratios at 10k nodes.

group, and connecting groups through a small number of global links. As shown in Fig. 2a, communication across groups is forced onto these limited inter-group paths once a job spans multiple groups, which quickly become bottlenecks. Consequently, communication performance becomes highly placement-sensitive, limiting structural consistency for multi-tenant AI training.

Expander. Expander topologies [9, 51, 53, 59], typically constructed using *random graph* techniques, rely on uniformly random connectivity to achieve small diameter. In Fig. 2b, this randomness means that even nearby nodes may lack direct links and must communicate via multi-hop paths through distant nodes. For AI training workloads, where communication is dominated by dense exchanges among nearby workers, this weak local connectivity forces traffic onto global paths and degrades synchronization efficiency.

2.4 Key Observation Behind LOONGX

LOONGX builds on the small-world effect [7, 40, 52], where introducing a small fraction of random shortcuts into a structured network can dramatically reduce diameter and average path length (APL). As shown in Fig. 3, both APL and diameter decrease rapidly as a small amount of randomness is introduced, but quickly saturate thereafter, with the fully random expander representing the lower bound. In particular, with dimension $w = 2$, allocating only 20% of uplinks to random shortcuts increases APL by just 5.1% compared to an expander, and this gap further shrinks to 3.4% at 30% randomness. Leveraging this property, allocating only a limited fraction of edges to random shortcuts can achieve expander-like diameter and near-optimal APL, while reserving the re-

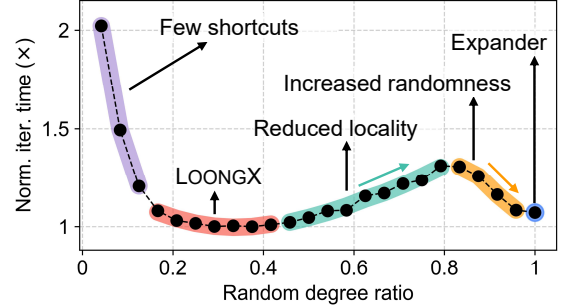


Figure 4: Design space of LOONGX.

maining degree for structured local connectivity.

Fig. 4 illustrates the relationship between topological randomness and iteration time on a 2.5k-node LOONGX topology. We run five concurrent DeepSeekV3 training jobs, each occupying 512 nodes; additional simulation details are provided in §5.1. The random degree ratio denotes the fraction of uplink ports assigned to random shortcuts.

At low randomness below 0.2, the limited number of shortcuts leads to large diameter and APL, causing even modest long-distance communication to traverse excessive hops and degrade performance. As randomness increases to the range of 0.2–0.4, the topology simultaneously achieves dense local connectivity and low network diameter, resulting in the lowest iteration time. When randomness further increases to 0.4–0.8, additional randomness provides little further reduction in diameter or APL, but increasingly disrupts structural locality and degrades short-range communication efficiency. At very high randomness above 0.8, locality is largely lost, but strong structural consistency reduces sensitivity to task placement, leading to a partial performance recovery that remains inferior to the optimal range of 0.2–0.4.

3 Design

In this section, we present the design of LOONGX, including its construction (§3.1), fundamental properties (§3.2), routing strategy (§3.3), and job placement (§3.4).

3.1 LOONGX Construction

While LOONGX is not an expander graph per se, we use expander constructions as a reference point to introduce and contextualize the LOONGX topology. Expander or random regular graphs can be defined as $EX(n, d)$, where n is the number of nodes and each node has d uplinks². A standard expander construction connects each node to d uniformly chosen peers while avoiding self-loops and duplicate edges. The left part of Fig. 5 shows an expander graph with $n = 36$ and $d = 5$, in which all edges are randomly generated. We refer to these edges as global edges to distinguish them from the structured local edges introduced in LOONGX.

Random shortcuts r . We denote LOONGX as $LX(n, d, r, w)$. Compared to an expander $EX(n, d)$, LOONGX introduces two

²Here, d denotes the number of uplink ports, rather than the total port count of a switch, for simplicity of exposition.

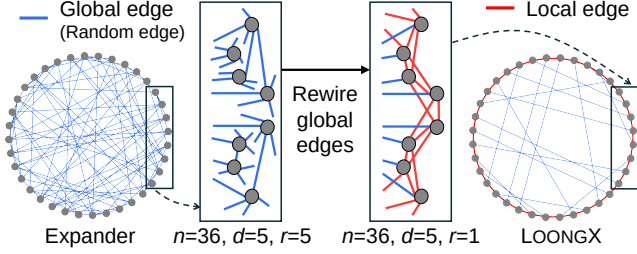


Figure 5: From Expander to LOONGX.

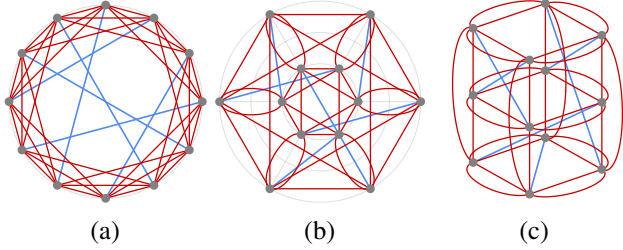


Figure 6: Node layouts of LOONGX ($n = 12$, $d = 7$, $r = 1$). Red: structured local edges; blue: random shortcuts. (a) $w = 1$, (b) $w = 2$, (c) $w = 3$.

additional parameters: the number of random edges r and the number of structured dimensions w . In particular, an expander corresponds to the special case $r = d$, where all d uplinks are used for random connections. Each node in LOONGX partitions its d ports into two components:

- r random edges providing global connectivity,
- $d - r$ structured local edges.

The random degree r controls global connectivity. As discussed in §2.4, we set r/d to 0.2–0.4, which is sufficient to achieve expander-like diameter while preserving strong local connectivity. Fig. 5 shows an example of transforming an expander into LOONGX. Starting from an expander with $n = 36$ and $d = 5$ ($r = 5$), LOONGX with $w = 1$ reduces the number of random edges to $r = 1$ and reallocates the remaining $d - r = 4$ ports to structured local connections.

Structured dimensions w . The $d - r$ local edges are organized into w independent cyclic dimensions. LOONGX’s local structure can be viewed as a generalized Torus: while a standard w -dimensional Torus connects each node only to its ± 1 neighbors in each dimension, LOONGX connects each node to multiple nearby neighbors, e.g., $\pm 1, \pm 2, \dots$, until the local ports in that dimension are used. Each dimension i is allocated d_i local ports, such that $\sum_{i=1}^w d_i = d - r$.

The choice of w determines how local connectivity is organized. Specifically, $d - r$ controls the overall strength of local connectivity, while w determines the geometric structure of the local region across multiple dimensions. Increasing w allows local regions to expand along multiple independent directions, offering greater flexibility in structured design space between fully structured and fully random topologies:

Fig. 6 shows a 12-node example with $w = 1, 2$, and 3. The $LX(n, d, r, w)$ parameterization defines a continuous design space between fully structured and fully random topologies:

- $r = d$, $LX(n, d, r, w)$ becomes $EX(n, d)$;

Algorithm 1 LOONGX CONSTRUCTION

Require:

- $\mathcal{R}$: set of nodes, $|\mathcal{R}| = n$;
- d : number of uplinks per node;
- r : number of random edges, $0 \leq r \leq d$;
- w : number of dimensions;
- $\{d_i\}_{i=1..w}$ with $\sum_{i=1}^w d_i = d - r$;
- $\mathcal{L}$: structured layout policy for w dimensions.

Ensure:

- $LX(n, d, r, w) = (V, E) \leftarrow$ simple undirected d -regular graph over $V = \mathcal{R}$.
- 1: $V \leftarrow \mathcal{R}; \quad E \leftarrow \emptyset$ // V denotes the vertices, and E denotes the edges.
// Step 1: build the $(d - r)$ -regular backbone (see Alg. 2 for details)
 - 2: $E_{\text{local}} \leftarrow \text{BUILDSTRUCTUREDBACKBONE}(V, d - r, \mathcal{L})$
// Step 2: add a shortcut layer (see Alg. 3 for details)
 - 3: $E_{\text{short}} \leftarrow \text{ADDSHORTCUTS}(V, r, E_{\text{local}})$
// Merge edges; resulting degree per node is $(d - r) + r = d$
 - 4: $E \leftarrow E_{\text{local}} \cup E_{\text{short}}$
 - 5: **return** $LX(n, d, r, w) = (V, E)$

- $r = 0$, $LX(n, d, r, w)$ reduces to a purely structured topology;
- $r = 0$ and $d/w = 2$, $LX(n, d, r, w)$ corresponds to a w -dimensional Torus.

LOONGX is constructed in two steps as outlined in Alg. 1, with the detailed pseudocode for its subroutines provided in Appx. C. First, BUILDSTRUCTUREDBACKBONE (Alg. 2) establishes the structured backbone using the $d - r$ local ports and the chosen w -dimensional layout, forming structured local connectivity. Second, ADDSHORTCUTS (Alg. 3) adds r random shortcut edges to each node via uniform sampling while avoiding duplicates and self-loops.

3.2 Properties of LOONGX

Low diameter. LOONGX combines a structured local backbone with a small number of random shortcuts. Our results in §2.4 show that when only 20% of uplinks are allocated to random shortcuts, the resulting topology achieves expander-like diameter, while its APL is only about 13% higher. As a result, LOONGX maintains short path lengths even at large scales, enabling efficient communication across many nodes.

Locality preservation. Unlike purely random topologies, LOONGX incorporates structured local connectivity that gives rise to strong locality. We quantify locality using the *clustering coefficient*³, defined as the probability that two neighbors of a node are also connected to each other. As shown in §5.2 and Table 4, LOONGX achieves a substantially higher clustering coefficient than Expander and Torus, and is comparable to that of Dragonfly, which exhibits strong locality through fully connected groups.

For fixed n and d , increasing the dimension w tends to reduce the clustering coefficient, as nodes become more sparsely connected in higher-dimensional structures. We formalize this relationship as Theorem 3.1, with the proof deferred to Appx. D.

³The clustering coefficient is defined for flat topologies and is not directly applicable to hierarchical designs such as Clos, ZCube, or Rail-optimized.

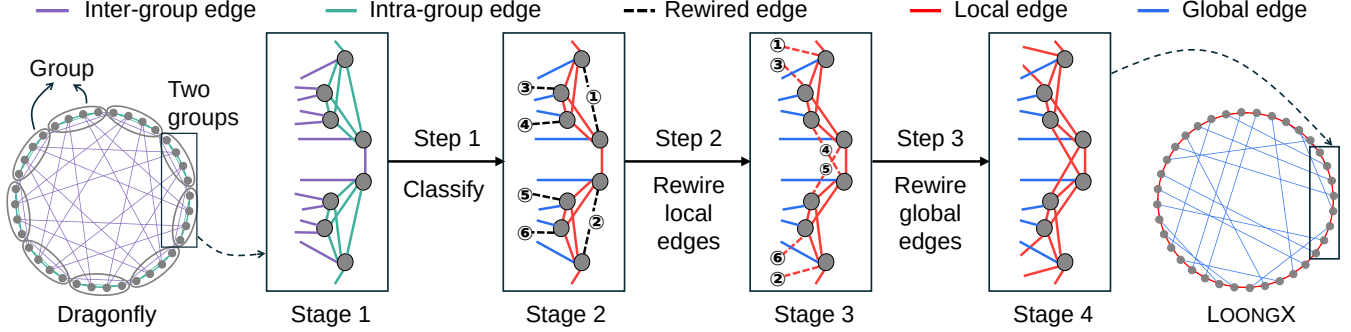


Figure 7: From Dragonfly to LOONGX.

Structural consistency. The LOONGX structure exhibits a uniform local geometry across the topology: each node connects to nearby neighbors through $d - r$ structured ports along w dimensions, and different parts of the topology share similar local connectivity patterns. As illustrated by the zoomed-in view of LOONGX in Fig. 5, every node has the same local connectivity structure. This uniformity allows jobs of different sizes to be placed on parts of the topology with similar geometric properties. Intuitively, LOONGX can be viewed as many similar local units, lightly connected by a small number of global shortcuts.

Path diversity. Structured local connectivity provides deterministic short paths that preserve locality for training traffic. Adding a fraction r of random shortcuts introduces long-range alternatives and increases path diversity, allowing traffic to spread across multiple routes under load and reducing concentration on individual bottlenecks.

3.3 Routing

We follow common practice for expander-like networks and use K-shortest path (**Ksp**) routing [37, 51, 53]. Ksp serves as a standard baseline to enumerate multiple available paths and does not introduce additional structure or optimization. Its role is solely to expose the path diversity inherent in the topology, rather than to create it.

Beyond Ksp, we also evaluate **Spraypoint** [9], the routing algorithm of RNG, the first production-scale flat datacenter network built on random graphs. Because LOONGX preserves a fraction of random shortcuts with expander-like global structure, Spraypoint applies naturally to LOONGX and offers a deployment-friendly alternative to Ksp; we report its performance in Fig. 16.

Under both routing setups, LOONGX’s locality-biased structure naturally serves local and global traffic differently. Communication confined to nearby nodes finds many short paths through the dense local backbone, while larger jobs spanning the cluster stay short via the random shortcuts. In both cases, the routing layer simply exposes this path diversity rather than creating it.

3.4 Job Placement

Placement overview. We evaluate LOONGX under three placement policies, all applied uniformly to every evaluated

topology. The default is a simple **greedy** policy that assigns each job to a block of nodes contiguous along LOONGX’s structured backbone and falls back to nearby blocks under fragmentation, used in all experiments unless otherwise specified. To verify that LOONGX’s benefits stem from its topology, we further evaluate two contrasting policies in §5.5: **random** placement scatters each job as a high-fragmentation stress test that disrupts locality and forces traffic onto the random shortcuts, while **topology-aware** placement serves as an upper reference.

Greedy placement. The greedy policy operates under two common situations. The first is contiguous block selection, used when a job can be assigned a single contiguous block of nodes. Placement selects a block of nodes that is contiguous along this backbone, with the block size chosen to match the job’s requirement, and the block can be extended incrementally along one or multiple dimensions to support jobs of different scales without predefined region boundaries. The second is non-contiguous block selection, used when fragmentation makes a fully contiguous block unavailable. In this case, placement follows the same backbone ordering and selects nodes from blocks that are adjacent along this ordering, assembling a limited number of such blocks until the job’s requirement is met, providing a simple and consistent fallback under fragmentation.

4 Deployment Considerations

Here we discuss deployment considerations for LOONGX, focusing on cabling and wiring (§4.1), incremental expansion (§4.2), cost analysis (§4.3), and failure handling (§4.4).

4.1 Cabling and Wiring

While LOONGX introduces a new connectivity structure, its cabling can be derived from existing topologies rather than built from scratch. In particular, LOONGX can be obtained by rewiring existing Expander and Dragonfly topologies.

Rewiring from an Expander. Starting from a random regular expander with degree d , we rewire $d - r$ edges per node to form structured local connections among physically nearby nodes, while retaining only r random edges for long-range connectivity (Fig. 5). The rewired edges follow fixed and repeatable local wiring patterns, creating dense local connec-

tivity while keeping most links short and spatially constrained. Compared to a fully random expander, this construction reduces cabling complexity by limiting the number of long cables, making the topology more amenable to practical deployment in large-scale AI clusters.

Rewiring from a Dragonfly. LOONGX can also be obtained by rewiring an existing Dragonfly topology. As illustrated in Fig. 7, we first classify Dragonfly links into dense intra-group edges and sparse inter-group edges. Links that already conform to the target LOONGX structure, including both local and long-range connections, are preserved, while the remaining links are designated as rewiring candidates. These candidate links are then reassigned to complete the structured local connectivity required by LOONGX. Any leftover links are finally repurposed as global edges to provide long-range connectivity. This workflow prioritizes the formation of LOONGX’s structured local connectivity while maximizing the reuse of existing links.

Avoiding spaghetti cabling. A common concern for random-graph topologies is spaghetti cabling, where logical random edges translate into arbitrary long-distance cables that are difficult to install and maintain. Recent production experience with RNG [9] shows that this concern can be addressed using passive ShuffleBoxes, which internally shuffle cable endpoints and expose planned panels to routers. LOONGX can apply the same cabling abstraction to its random shortcut layer, allowing the topology to scale without requiring arbitrary router-to-router cabling.

4.2 Incremental Expansion

A key challenge in large-scale AI clusters is enabling incremental expansion without disruptive rewiring. LOONGX addresses this challenge by decoupling local structural growth from global random connectivity: new nodes are integrated through structured local dimensions, while only a small number of random shortcuts are adjusted to preserve global properties. The detailed procedure of incremental expansion is described in Alg. 4 in Appx. E.

Structured local expansion. Expander topologies are difficult to grow incrementally because each node uses all d ports for global connectivity, so adding a new node often requires reconfiguring many existing random links. In contrast, LOONGX explicitly encodes locality through structured dimensions that define stable local neighborhoods. This structure allows new nodes to be integrated by extending these dimensions with only localized rewiring, while leaving the rest of the topology unchanged.

Incremental global shortcut assignment. To preserve global connectivity during expansion, each node in LOONGX maintains only r random shortcuts. When adding a new node, incorporating it into the global structure requires modifying only a small number of long-range edges, rather than reconfiguring all d connections. Assigning r new shortcuts is sufficient to maintain low diameter and short path length.

4.3 Cost Analysis

From a cost perspective, LOONGX inherits the advantages of Expander. LOONGX uses the same number of switches and per-node degree as an Expander, differing only in how a fraction of links are rewired to form structured local connectivity. As a result, the total number of switches, inter-switch links, and overall cabling volume remain comparable to those of an Expander, allowing LOONGX to retain its well-established cost benefits, namely requiring only about 80–85% of the switching hardware of hierarchical topologies under the same host scale [9, 53].

The detailed price breakdown is shown in Table 6, with pricing obtained from prior work [32, 56]. We primarily adopt the MixNet [32] cost model, and additionally reference raw ATOP [60] cost data where available. When applied to the same topologies, the two models differ by only about 10–20% in absolute cost estimates due to pricing assumptions, indicating that our cost estimation closely reflects the underlying cost trends reported in prior work. Across all evaluated scales (Table 4), LOONGX incurs comparable cost to Expander, consistent with their identical switch counts and cabling requirements, while ZCube, which introduces an additional structural layer, consistently incurs higher cost. Overall, these results show that LOONGX introduces no additional baseline cost.

4.4 Failure Handling

Local failover via structured connectivity. LOONGX’s structured backbone provides multiple deterministic directions between nearby nodes. Under local link or node failures, Ksp selects alternative paths that avoid failed components, preserving locality with only a modest increase in hop count.

Global failover via random shortcuts. Each node also maintains a small number of random shortcuts connecting distant regions. These shortcuts provide independent long-range alternatives, allowing flows to bypass failed regions without incurring long detours.

Path stretch under failures. By combining structured local alternatives with random global shortcuts, LOONGX limits path stretch under common failure scenarios. As shown in Fig. 19, even under a 10% link failure rate, the average path length increases by less than 2%.

5 Evaluation

In this section, we cover simulation setup (§5.1), followed by topology characteristics (§5.2), isolated AI workload performance (§5.3), multi-tenant AI workload performance (§5.4), and evaluation under various settings (§5.5).

5.1 Simulation Setup

Simulation framework. We use htsim [10], a discrete-event, packet-level simulator widely adopted for evaluating data center networks [21, 28, 37]. Its packet-level granularity enables high-fidelity modeling of network dynamics. However, the native simulator only supports non-AI traffic, where flows

are independent of one another, and does not capture the flow dependencies in AI training workloads. To address this limitation, we extend htsim with a directed acyclic graph (DAG) scheduler that models task-dependency graphs in AI training. More details are provided in Fig. 21 in Appx. B.

LOONGX parameter setting. Recall that LOONGX is modeled as $LX(n, d, r, w)$. We evaluate networks with $n = 128, 320, 640$, and 1280 switches, corresponding to approximately 1k, 2.5k, 5k, and 10k nodes, respectively. Unless otherwise specified, we use a default configuration of $d = 24$ and $r = 6$, corresponding to 25% random shortcuts, and set the dimension to $w = 1$. In §5.5, we further explore the design space by varying d , r , and w . More details on the network settings of LOONGX and baselines are provided in Table 4.

Topology baselines. We compare LOONGX with two representative classes of network architectures. The first class consists of hierarchical topologies, namely the industry-standard Clos [3], Rail-optimized [1], and ZCube [60], the latter designed for large model training. The second class consists of flat topologies, namely Torus [26], Dragonfly [27] commonly used in HPC systems, and Expander [22] derived from random-graph theory. We evaluate networks with 1k, 2.5k, 5k, and 10k nodes, with configurations summarized in Table 4.

Workloads. We evaluate five representative model configurations: DeepSeekV3-671B-A37B [33], Qwen3-235B-A22B [61], GLaM-1.2T-A97B [15], DeepSpeedMoE-368B-A63B [46], and GPT3-175B-A175B [11]. These workloads are generated using the AICB library in SimAI [57]. Operator-level computation is profiled on a 16-nodes GPU cluster with four DGX H200 GPUs per node and 141 GB memory each, and communication traces are derived from model specifications and parallelization strategies and scaled to different network sizes. Detailed configurations are provided in Table 2 (Appx. A). In addition, we include a dense all-to-all workload, where each node sends 1 GB to every other node. We also evaluate generic data center traffic (non-AI workloads), with results reported in Appx. H.

Routing policy. We adopt routing strategies based on topology type. For hierarchical topologies, namely Clos, Rail-optimized, and ZCube, we enable Equal-Cost Multi-Path (ECMP) [24] to distribute traffic across available paths. For flat topologies, namely LOONGX, Torus, Dragonfly, and Expander, we use Ksp [51, 53] with the 8 shortest paths to exploit path diversity and balance load. By default, each flow uses a single path. Beyond Ksp, we also evaluate Spraypoint [9] on LOONGX as an alternative routing algorithm. For load balancing, we further consider finer-grained packet-level random spray and round-robin (RR) spray [36].

Transport protocols. We use NDP [21] as the default transport protocol and compare it with Bolt [6], HPCC [30], and ExpressPass [14]. Unless otherwise specified, all links operate at 100 Gbps with an MTU of 1.5 KB. Table 3 summarizes the detailed settings.

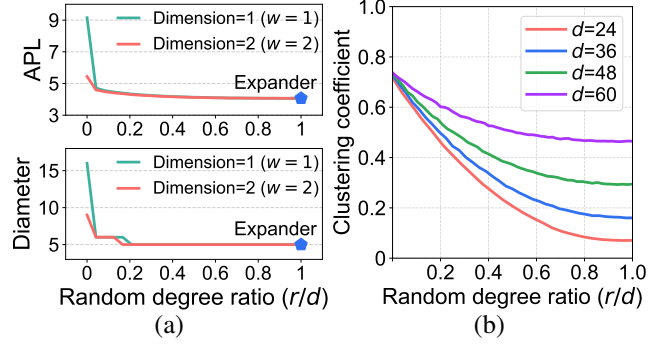


Figure 8: (a) APL, diameter, and (b) Clustering coefficient under varying random degree ratios in 2.5k nodes.

5.2 Topology Characteristics

We evaluate the topological properties of LOONGX to provide context for the subsequent results. We consider networks at a scale of approximately 2.5k-node and report average path length (APL), diameter, and clustering coefficient.

APL and diameter. Fig. 8a reports the APL and diameter for networks with $d = 24$ uplinks. The random degree ratio r/d captures the fraction of uplink ports assigned to random shortcuts; $r/d = 1$ yields an Expander, while $r/d = 0$ corresponds to purely local connectivity. Both metrics improve rapidly with the introduction of a small fraction of random shortcuts. Under $w = 1$, the Expander achieves an APL of 4.07, while LOONGX with $r/d = 0.2$ increases APL only to 4.34 (6.6%). With 30% random shortcuts, the APL overhead further drops to 4.4%, beyond which additional shortcuts yield diminishing returns. A similar trend is observed for diameter: LOONGX matches the Expander diameter of 5 with only 20% random shortcuts. Overall, LOONGX achieves Expander-like topology properties with a small random budget. We further report the hop count distribution in Appx. F.

Clustering coefficient. Fig. 8b reports the clustering coefficient under four uplink configurations ($d \in 24, 36, 48, 60$) with $w = 1$. As d increases, the clustering coefficient rises, reflecting stronger local connectivity enabled by higher-radix switches. In contrast, the clustering coefficient decreases monotonically with the random degree ratio, indicating that random shortcuts progressively disrupt local triangles and weaken structured locality. For example, with $d = 24$, an Expander exhibits a clustering coefficient of only 0.07, whereas LOONGX achieves 0.45 with $r/d = 0.2$, a $6.4\times$ improvement. Additional results comparing LOONGX with flat topologies are summarized in Table 4, further confirming its advantage in preserving dense local structure.

5.3 Isolated AI Workload Performance

We evaluate the performance under isolated single-job training workloads. Fig. 9 reports the iteration time across cluster sizes of 1k, 5k, and 10k nodes. We also evaluate a dense model, GPT3, in Appx. G, which exhibits higher tensor parallel traffic than the models discussed here.

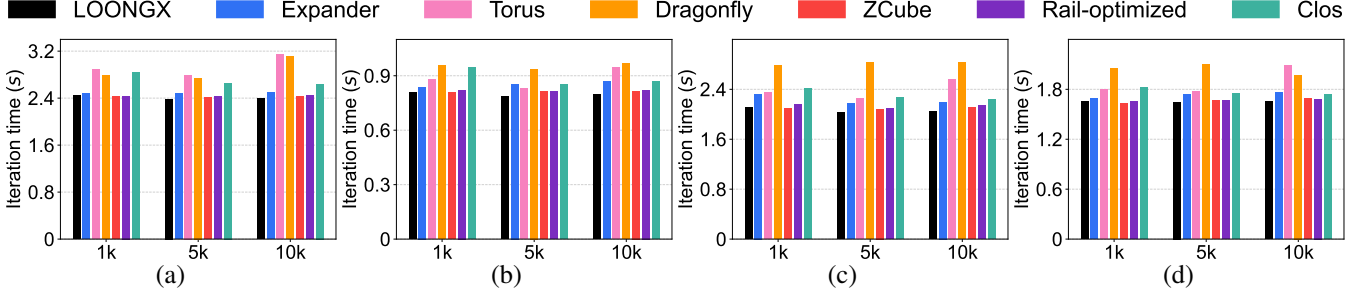


Figure 9: Iteration time across different AI cluster scales under isolated training workloads: (a) DeepSeekV3, (b) GLaM, (c) DeepSpeedMoE, and (d) Qwen3.

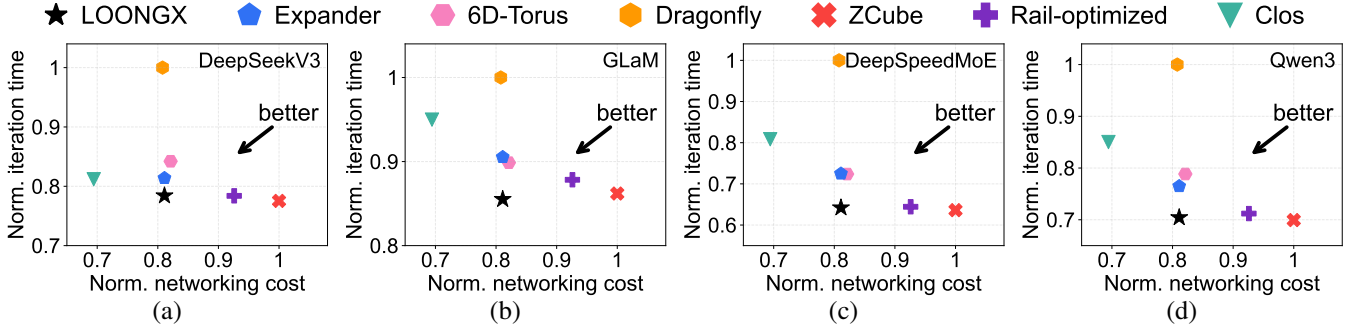


Figure 10: Iteration time vs. networking cost for concurrent workloads in a 2.5k-node multi-tenant AI cluster: (a) DeepSeekV3, (b) GLaM, (c) DeepSpeedMoE, and (d) Qwen3.

Compared with flat topologies. We first compare LOONGX with flat topologies, i.e., Expander, Torus, and Dragonfly. Across all evaluated workloads and scales, LOONGX consistently outperforms these baselines. Dragonfly performs the worst, incurring up to a 39.3% higher iteration time than LOONGX under the DeepSpeedMoE workload at the 5k-node scale, due to its reliance on sparse inter-group links that quickly saturate under global communication. Torus exhibits a 5.4%–31% slowdown, primarily caused by its larger network diameter. Expander shows a smaller but consistent gap, with iteration time increasing by 10.2% at 1k nodes and 7.6% at 10k nodes for DeepSpeedMoE, reflecting its lack of explicit locality. By combining dense local connectivity with efficient global shortcuts, LOONGX better matches the communication patterns of large-scale AI training.

Compared with hierarchical topologies. We compare LOONGX with hierarchical designs, ZCube, Rail-optimized, and Clos. ZCube and Rail-optimized achieve performance close to LOONGX, but at $1.1\text{--}1.25\times$ higher networking cost (Table 4). This overhead arises because ZCube adds an extra recursive layer with multi-port NICs, while Rail-optimized over-provisions a non-blocking, Clos-like fabric. In contrast, Clos suffers from congestion because of limited bandwidth at upper tiers, incurring up to 16.6% higher iteration times than LOONGX at the 1k-node scale across the four evaluated models. Overall, LOONGX achieves performance comparable to costly optimized hierarchical designs while avoiding the bandwidth bottlenecks of Clos.

5.4 Multi-Tenant AI Workload Performance

Iteration time and networking cost trade-off. We evaluate the trade-off between iteration time performance and networking cost under multi-tenant training, where four heterogeneous models execute concurrently and share network resources. Component cost assumptions follow prior work [32, 56] (Table 6, assuming 100 Gbps setting), and network configurations are detailed in Table 4.

Better performance than flat topologies. Fig. 10 shows normalized iteration time versus normalized networking cost for a 2.5k-node cluster. Flat topologies, including Expander, Dragonfly, and Torus, have comparable network cost to LOONGX since all are flat designs with similar switch and link counts. However, Expander incurs up to 13% higher iteration time (DeepSpeedMoE), Dragonfly performs significantly worse with a 55.7% increase, and Torus shows a 11.9% slowdown for Qwen3. These gaps arise from structural limitations: Expander lacks locality and forces local traffic to detour through distant nodes, Torus suffers from large hop counts across blocks, and Dragonfly is constrained by sparse inter-group bandwidth. By contrast, LOONGX combines dense local connectivity with a small fraction of random shortcuts, achieving both locality and low diameter, and consistently outperforming these flat topologies in multi-tenant training.

Lower cost than hierarchical topologies. ZCube and Rail-optimized achieve performance comparable to LOONGX, but both incur higher networking cost to sustain high throughput. Specifically, under DeepSeekV3 at the 2.5k-node

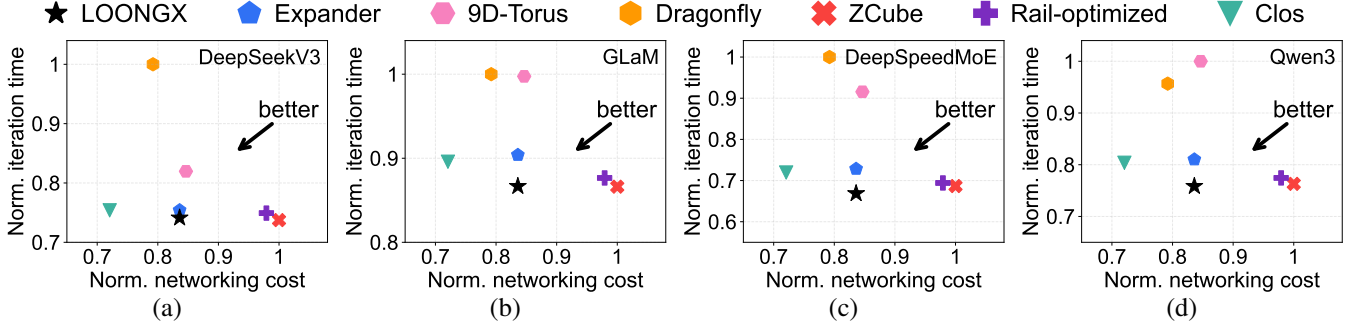


Figure 11: Iteration time vs. networking cost for concurrent workloads in a 10k-node multi-tenant AI cluster: (a) DeepSeekV3, (b) GLaM, (c) DeepSpeedMoE, and (d) Qwen3.

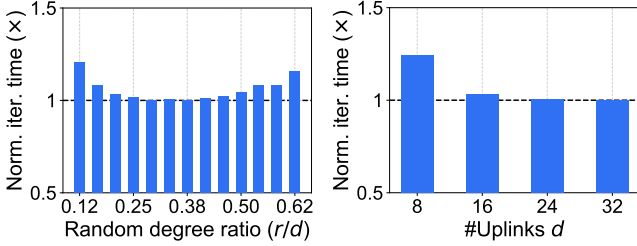


Figure 12: Impact of the number of random edges r . Figure 13: Impact of the number of uplinks d .

scale, ZCube attains only 1.1% lower iteration time than LOONGX while requiring 23.3% higher network cost, and Rail-optimized achieves nearly identical iteration time while requiring 14.2% higher network cost. In contrast, LOONGX achieves similar performance through architectural design rather than hardware over-provisioning: it confines dense communication to a structured local backbone while maintaining sufficient global connectivity. Therefore, LOONGX matches optimized hierarchical baselines at lower cost, yielding a more favorable performance–cost trade-off.

Better performance than Clos. Clos architectures exhibit a fundamental cost–performance trade-off: oversubscription lowers cost but induces severe congestion. With oversubscription = 2, Clos reduces network cost by 14%, yet increases iteration time by 25.9% for DeepSpeedMoE and 20.7% for Qwen3. LOONGX sustains high performance without incurring the congestion penalties of oversubscribed Clos, achieving a more favorable trade-off. Fig. 11 presents results for a 10k-node cluster, where LOONGX maintains performance comparable to that in Fig. 10, indicating that its topological properties scale effectively with cluster size.

5.5 Performance under Various Settings

We next evaluate different design aspects of LOONGX under diverse settings. Unless otherwise specified, all experiments use a 2.5k-node LOONGX topology with AI workloads.

Varying number of random edges r . Fig. 12 shows the impact of varying the number of random edges r with a fixed uplink degree $d = 24$. As r increases from zero, the iteration time decreases, whereas too few random edges lead to a large

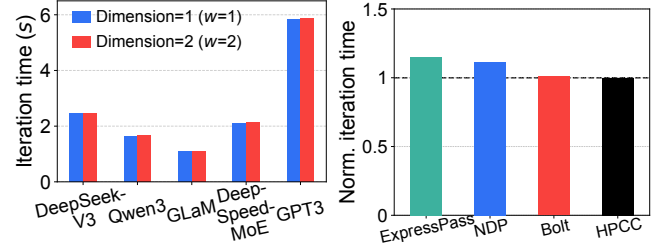


Figure 14: Iteration time under varying dimension w . Figure 15: Iteration time under different transport protocols.

network diameter (Fig. 8a), causing even a small amount of long-distance traffic to become a bottleneck due to excessive hop counts. A random degree ratio in the range of 20%–40% yields the lowest iteration time, where LOONGX balances dense local connectivity with sufficient random shortcuts to maintain a low diameter for global communication. When the random degree ratio exceeds 40%, performance degrades as excessive randomization weakens structural locality and inflates the path length of local communication, as illustrated in the right part of Fig. 8.

Varying number of uplinks d . We fix the random degree ratio at 25% and vary the number of uplinks d , with the number of random edges scaling accordingly. As shown in Fig. 13, increasing d reduces iteration time, yielding a 17% improvement when d increases from 8 to 16 due to higher path diversity and fan-out. Further increasing d from 24 to 32 provides diminishing gains, indicating saturation once sufficient connectivity is reached.

Varying dimension w . Fig. 14 compares five AI workloads under dimensions $w = 1$ and $w = 2$. Across all workloads, 2D-LOONGX achieves performance comparable to the 1D baseline, with iteration time increases of 0.53% for GLaM, 1.43% for Qwen3, 1.22% for DeepSpeedMoE, 0.67% for DeepSeekV3, and 0.64% for GPT3. A breakdown of computation and communication suggests that the additional network overhead from the higher-dimensional topology is small, implying minimal performance impact when extending LOONGX to higher dimensions.

Impact of transport protocols. Fig. 15 evaluates interaction between transport protocols and the LOONGX topology. Bolt

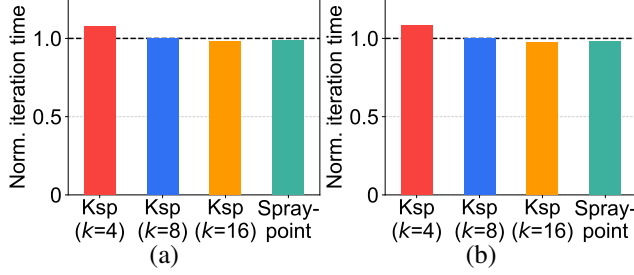


Figure 16: Iteration time under different routing policies for (a) 1k-node and (b) 10k-node topology.

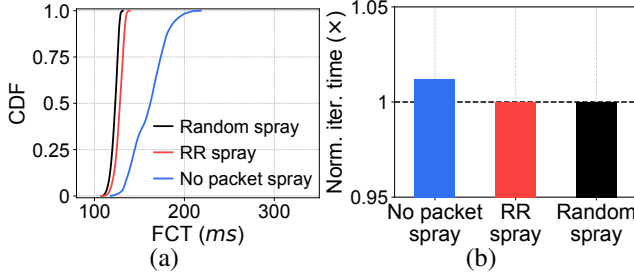


Figure 17: FCTs under different load balancing granularities for (a) all-to-all traffic and (b) the DeepSeekV3 workload.

and HPCC achieve the best performance, while ExpressPass and NDP incur 15% and 11% higher iteration times, respectively. This behavior reflects protocol design assumptions: HPCC and Bolt rely on fine-grained, feedback-driven congestion control and adapt well to flat topologies adopting Ksp, whereas NDP and ExpressPass are tailored to Clos-style hierarchical networks and are less aligned with flat connectivity.

Impact of routing algorithms. Fig. 16 compares different routing algorithms on LOONGX. We use Ksp ($k=8$) as the baseline. At 1k nodes, Ksp ($k=4$) increases iteration time by 7.9% due to limited path diversity, while Ksp ($k=16$) and Spraypoint reduce iteration time by 1.9% and 1.5%, respectively. Spraypoint, a practical routing scheme suitable for deployment, achieves performance close to Ksp ($k=16$), suggesting that LOONGX can exploit path diversity without relying on a large Ksp path set. The 10k-node results show a similar trend.

Impact of load balancing granularity. We evaluate three load balancing granularities on two traces: dense all-to-all traffic, where each node sends 1 GB to every other node (Fig. 17a), and the DeepSeekV3 workload (Fig. 17b). The default no packet spray policy performs the worst among all schemes, exhibiting 57% and 50% higher 99%-ile FCT than random and RR spray, respectively, under dense all-to-all traffic. For the DeepSeekV3 workload with sparser communication patterns, the performance gains from packet spray are reduced. Overall, packet spray improves network performance by enhancing load balancing, at the cost of potential packet reordering overhead.

Robustness to placement. In Fig. 18, LOONGX remains robust under both topology-aware and random placements.

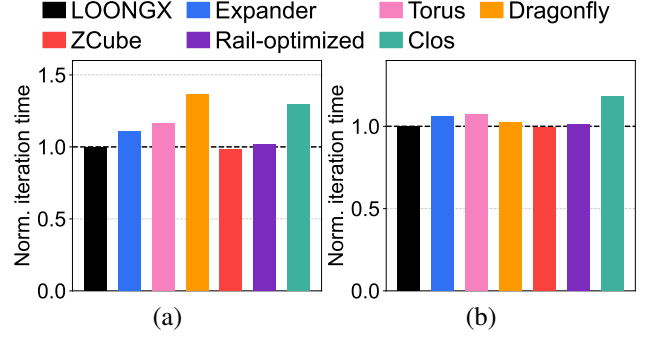


Figure 18: Normalized iteration time under (a) topology-aware placement and (b) random placement.

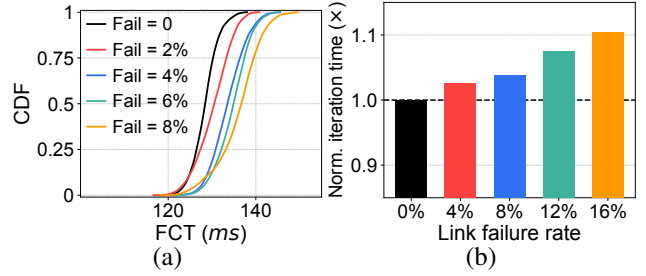


Figure 19: (a) FCT distributions for the all-to-all workload and (b) Normalized iteration time for the DeepSeekV3 workload under varying link failure rates.

Under topology-aware placement, LOONGX is within 1.5% of ZCube and Rail-optimized, while outperforming Expander, 6D-Torus, Dragonfly, and Clos by 10.6–36.7%. Under random placement, which maximally fragments locality, LOONGX stays nearly tied with ZCube and Rail-optimized and remains competitive with Expander at substantially lower cost, while still outperforming the remaining baselines. These results indicate that LOONGX’s benefit does not stem from a topology-specific placement heuristic: its structured local connectivity is leveraged when locality is preserved, while its random shortcuts keep paths short once placement becomes fragmented.

Robustness to failures. Fig. 19 evaluates the robustness of LOONGX under varying link failure rates for dense all-to-all traffic and a DeepSeekV3 training trace, with Fig. 19a and Fig. 19b reporting the FCT distributions and iteration time, respectively. LOONGX remains stable under link failures: for all-to-all traffic, the 99%-ile FCT increases by 5.1% at a 4% failure rate and 7.5% at a 8% failure rate, while for the sparser AI workload, iteration time increases by 2.6% at a 4% failure rate and 3.8% at a 8% failure rate. This robustness arises from dense local connectivity combined with random shortcuts, which preserve short paths while enabling efficient detours around failures.

We also compute the APL under a 10% link failure rate, denoted as APL_{fail} , as shown in Table 4. LOONGX incurs less than a 2% path length stretch, further confirming the robustness observed in Fig. 19.

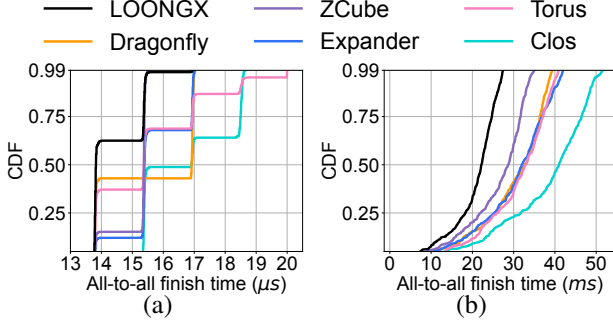


Figure 20: All-to-all traffic finish time for (a) 10KB and (b) 10MB flows.

6 Prototype Testbed

Testbed setup. We validate LOONGX on a small-scale testbed and implement other baselines on the same setup to cross-validate simulation results. The testbed consists of an EdgeCore DCS810 switch with Intel Tofino2 and eight servers equipped with Mellanox ConnectX-5 dual-port 100Gbps NICs. To emulate a 256-nodes topology, we leverage the Tofino2 programmable data plane to virtualize 32–64 logical switches, while using the eight servers to virtualize 256 end hosts. We further use the packet replication engine on Tofino2 to amplify traffic on inter-switch links, which are configured to operate at 100Gbps. Detailed topology settings are provided in Table 5.

Workloads. We synthesize all-to-all workloads based on an MoE architecture with two data parallel groups (DP=2) and eight pipeline parallel groups (PP=8), each containing eight experts (EP=16). All-to-all communication occurs within each PP group, where each expert follows a Top- k routing strategy ($k = 4$) by randomly selecting four target experts from the same group to distribute data. The synthesized traffic includes both short (10KB) and long (10MB) flows.

Performance of short flows. Fig. 20a shows that under short flows, LOONGX consistently outperforms the baselines. The stair-step pattern reflects the discrete hop counts traversed by each flow. Owing to LOONGX’s dense local connectivity, 98% of the flows complete within one or two hops. Meanwhile, flows in Clos frequently traverse multiple stages, resulting in inherently higher hop counts. Torus exhibits the highest 99%-ile all-to-all finish time, primarily due to its larger network diameter.

Performance of long flows. Fig. 20b shows smoother curves for long flows, as their performance is dominated by path sharing and available bandwidth rather than packet-level effects in short flows. Therefore, the curves largely track the average hop count of each flow. By combining strong local connectivity with a small number of global shortcuts, LOONGX minimizes hop counts and achieves the best performance. At the 99%-ile all-to-all finish time, LOONGX outperforms ZCube by 8.4%, with 53%, 43%, 49%, and 88% lower finish times than Expander, Dragonfly, Torus, and Clos, respectively. ZCube benefits from its multi-NIC design, which helps reduce

path sharing. Expander delivers moderate performance but is constrained by weak local clustering, while Dragonfly is limited by inter-group bottlenecks despite dense intra-group connectivity. Torus benefits from improved locality under long flows, whereas Clos performs worst due to its longer average path lengths across flows.

7 Related Work

Training-centric network architectures. Large-scale AI training clusters commonly rely on Clos-like fabrics [17, 35, 44]. To exploit hierarchical bandwidth, rail-optimized and rail-only designs align collectives to preferred communication rails, primarily benefiting structured all-reduce workloads [1, 55]. Google adopts Torus-like topologies in its training supercomputers to match accelerator layouts and provide resiliency [26, 63]. Beyond fixed templates, per-job topology generation has been explored but is challenging to apply in shared multi-tenant environments [56]. More recent training-aware topologies, such as ZCube [60] and HammingMesh [23], further explore the alignment between AI training communication patterns and network structure.

Data center networks. Modern data center networks predominantly use Clos fabrics to provide uniform bandwidth for cloud workloads [2, 3, 19, 20, 34, 43]. To reduce path lengths, Dragonfly-style designs organize routers into groups with high global connectivity to achieve low diameter [27, 50]. Expander-based topologies approximate minimum diameter graphs through irregular connectivity, including Jellyfish and Xpander [5, 22, 51, 53, 59]. Most recently, RNG demonstrated the first production flat datacenter network based on random graphs, making random graph topologies practical at hyperscale [9]. In parallel, optical and hybrid architectures support dynamically reconfigurable connectivity, ranging from early electrical-optical designs to microsecond-scale circuit switching and fully optical fabrics [8, 13, 16, 18, 31, 37, 38, 42, 54]. These architectures are primarily designed for cloud workloads with short-lived and randomized flows. Therefore, they fall short when applied to AI training communication, which is long-running, collective-heavy, and highly localized.

8 Conclusion

In this paper, we revisit the role of randomness in network design for multi-tenant AI training. We show that training communication is largely localized, and that uniformly random expander-style connectivity can over-allocate long-range links at the expense of local bandwidth. LOONGX instead combines structured local neighborhoods with a small number of random shortcuts, achieving low network diameter while preserving strong locality. More broadly, LOONGX offers a new perspective on AI training networks, where structured locality and limited randomness are co-designed rather than treated as competing choices, opening new directions for topology and system co-design in future AI clusters.

References

- [1] Rail-optimized topology. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>.
- [2] Saksham Agarwal et al. Harmony: A congestion-free datacenter architecture. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 329–343, 2024.
- [3] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review*, 38(4):63–74, 2008.
- [4] Mohammad Alizadeh, Albert Greenberg, David A Maltz, et al. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 63–74, 2010.
- [5] Daniel Amir, Nitika Saran, et al. Shale: A practical, scalable oblivious reconfigurable network. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 449–464, 2024.
- [6] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkkipati. Bolt: Sub-rtt congestion control for ultra-low latency. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 219–236, 2023.
- [7] Chen Avin, Robert Elsässer, Aleksander Figiel, Darya Melnyk, and Stefan Schmid. Demand-aware small-world networks on clustered demands. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025)*, pages 28–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2026.
- [8] Hitesh Ballani, Paolo Costa, et al. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 782–797, 2020.
- [9] Giacomo Bernardi, Ratul Mahajan, C. Seshadhri, et al. Rng: Flat datacenter networks at scale. *arXiv preprint arXiv:2604.15261*, 2026.
- [10] Broadcom. htsim. <https://github.com/Broadcom/csg-htsim>, 2023.
- [11] Tom Brown, Benjamin Mann, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [12] Jiamin Cao et al. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 1–15, 2024.
- [13] Kai Chen, Ankit Singla, et al. Osa: An optical switching architecture for data center networks with unprecedented flexibility. *IEEE/ACM Transactions on Networking*, 22(2):498–511, 2013.
- [14] Inho Cho, Keon Jang, and Dongsu Han. Credit-scheduled delay-bounded congestion control for datacenters. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 239–252, 2017.
- [15] Nan Du, Yanping Huang, et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*, pages 5547–5569. PMLR, 2022.
- [16] Nathan Farrington, George Porter, et al. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 339–350, 2010.
- [17] Adithya Gangidi, Rui Miao, Shengbao Zheng, et al. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, 2024.
- [18] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, et al. Projector: Agile reconfigurable data center interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 216–229, 2016.
- [19] Albert Greenberg, James R Hamilton, et al. V12: A scalable and flexible data center network. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*, pages 51–62, 2009.
- [20] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, et al. Bcube: a high performance, server-centric network architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*, pages 63–74, 2009.
- [21] Mark Handley, Costin Raiciu, et al. Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 29–42, 2017.
- [22] Vipul Harsh, Sangeetha Abdu Jyothi, Inderdeep Singh, and P Godfrey. Expander datacenters: From theory to practice. *arXiv preprint arXiv:1811.00212*, 2018.
- [23] Torsten Hoeffler, Tommaso Bonato, et al. Hammingmesh: A network topology for large-scale deep learning. *Communications of the ACM*, 67(12):97–105, 2024.
- [24] Christian Hopps. Analysis of an equal-cost multi-path algorithm. Technical report, 2000.

- [25] Yanping Huang, Youlong Cheng, Ankur Bapna, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.
- [26] Norm Jouppi et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*, pages 1–14, 2023.
- [27] John Kim, William J Dally, Steve Scott, and Dennis Abts. Technology-driven, highly-scalable dragonfly topology. *ACM SIGARCH Computer Architecture News*, 36(3):77–88, 2008.
- [28] Jialong Li, Haotian Gong, et al. Uniform-cost multi-path routing for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 433–448, 2024.
- [29] Shen Li, Yanli Zhao, Rohan Varma, et al. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704*, 2020.
- [30] Yuliang Li, Rui Miao, Hongqiang Harry Liu, et al. Hpc: High precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication*, pages 44–58. 2019.
- [31] Cong Liang, Xiangli Song, et al. Negotiator: Towards a simple yet effective on-demand reconfigurable data-center network. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 415–432, 2024.
- [32] Xudong Liao, Yijun Sun, Han Tian, et al. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 554–574, 2025.
- [33] Aixin Liu, Bei Feng, Bing Xue, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [34] Yunpeng James Liu, Peter Xiang Gao, Bernard Wong, and Srinivasan Keshav. Quartz: a new design element for low-latency dcns. *ACM SIGCOMM Computer Communication Review*, 44(4):283–294, 2014.
- [35] Jie Lu, Jiaqi Gao, Fei Feng, et al. Alibaba stellar: A new generation rdma network for cloud ai. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 453–466, 2025.
- [36] Sarah McClure, Sylvia Ratnasamy, and Scott Shenker. Load balancing for ai training workloads. *arXiv preprint arXiv:2507.21372*, 2025.
- [37] William M Mellette, Rajdeep Das, et al. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1–18, 2020.
- [38] William M Mellette, Rob McGuinness, et al. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 267–280, 2017.
- [39] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, et al. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM symposium on operating systems principles*, pages 1–15, 2019.
- [40] Van Nguyen and Chip Martel. Analyzing and characterizing small-world graphs. In *Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 311–320, 2005.
- [41] Jonathan Perry, Amy Ousterhout, Hari Balakrishnan, et al. Fastpass: A centralized "zero-queue" datacenter network. In *Proceedings of the 2014 ACM conference on SIGCOMM*, pages 307–318, 2014.
- [42] George Porter, Richard Strong, Nathan Farrington, et al. Integrating microsecond circuit switching into the data center. *ACM SIGCOMM Computer Communication Review*, 43(4):447–458, 2013.
- [43] Leon Poutievski, Omid Mashayekhi, Joon Ong, et al. Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 66–85, 2022.
- [44] Kun Qian, Yongqing Xi, et al. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 691–706, 2024.
- [45] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1403–1420, 2024.
- [46] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, et al. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*, pages 18332–18346. PMLR, 2022.

- [47] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.
- [48] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. Inside the social network’s (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 123–137, 2015.
- [49] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [50] Alexander Shpiner, Zachy Haramaty, et al. Dragonfly+: Low cost topology for scaling datacenters. In *2017 IEEE 3rd International Workshop on High-Performance Interconnection Networks in the Exascale and Big-Data Era (HiPINEB)*, pages 1–8. IEEE, 2017.
- [51] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P Brighten Godfrey. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 225–238, 2012.
- [52] Frank W Takes and Walter A Kusters. Determining the diameter of small world networks. In *Proceedings of the 20th ACM international conference on Information and knowledge management*, pages 1191–1196, 2011.
- [53] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International on Conference on emerging Networking EXperiments and Technologies*, pages 205–219, 2016.
- [54] Guohui Wang, David G Andersen, et al. c-through: Part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 327–338, 2010.
- [55] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, et al. How to build low-cost networks for large language models (without sacrificing performance)? *arXiv e-prints*, pages arXiv–2307, 2023.
- [56] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, et al. Topoopt: Optimizing the network topology for distributed dnn training. *arXiv preprint arXiv:2202.00433*, 2022.
- [57] Xizheng Wang, Qingxu Li, et al. {SimAI}: Unifying architecture design and performance tuning for {Large-Scale} large language model training with scalability and precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 541–558, 2025.
- [58] Qizhen Weng, Wencong Xiao, et al. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 945–960, 2022.
- [59] Yiting Xia, Xiaoye Steven Sun, et al. A tale of two topologies: Exploring convertible data center network architectures with flat-tree. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 295–308, 2017.
- [60] Zihan Yan, Dan Li, Li Chen, et al. From atop to zcube: Automated topology optimization pipeline and a highly cost-effective network topology for large model training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, pages 861–881, 2025.
- [61] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [62] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference*, pages 78–85, 2017.
- [63] Yazhou Zu et al. Resiliency at scale: Managing google’s tpuv4 machine learning supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 761–774, 2024.

Table 2: Model configurations.

Config	DeepSeekV3	Qwen3	GLaM	DeepSpeedMoE	GPT3
<i>Layers</i>	61	94	64	96	96
<i>FFN_H</i>	2048	1536	32768	2048	49152
<i>L</i>	4096	4096	1024	2048	2048
<i>H</i>	18432	4096	8192	12288	12288
<i>Heads</i>	128	64	128	96	96
<i>TP</i>	1	1	1	1	8
<i>PP</i>	8	8	8	8	8
<i>EP</i>	64	32	32	32	–
<i>DP</i>	$World_size / (TP \times PP \times EP)$				
<i>Experts</i>	256	128	64	64	–
<i>Micro_batch</i>	1	1	1	1	1
<i>Global_batch</i>	$DP \times Micro_batch \times Batch_num$				

Appendix

A Model Configurations

Table 2 summarizes the configurations of the large language models used in our evaluation in SimAI’s AICB. *Layers* denote the number of Transformer blocks. *H* is the hidden dimension, and *Heads* is the number of attention heads. *TP*, *PP*, *EP*, and *DP* denote the sizes of tensor parallelism, pipeline parallelism, expert parallelism, and data parallelism. *FFN_H* represents the hidden size of the feed-forward network (per expert for MoE models). *L* is the maximum sequence length. *Experts* specifies the number of experts per MoE layer. We additionally report *Micro_batch*, which is the per-GPU batch size, and *Global_batch*, computed as $DP \times Micro_batch \times Batch_num$ (set to 8 in our configuration), reflects the total effective batch size with gradient accumulation.

To generate realistic AI training traffic, we align *World_size*⁴ with the simulated cluster topology and configure the parallelism degrees to saturate available resources. The resulting traces explicitly capture communication from DP gradient synchronization, PP stage communication across forward and backward passes, EP token routing and expert aggregation, and TP sharded computation. We also account for TP traffic explicitly: although most TP communication occurs within a node over high-bandwidth links, cross-node tensor sharding can arise when model memory requirements exceed per-node capacity, adding additional load to the network that is reflected in our traces.

B Enhance htsim for AI Workloads

While the native *htsim* excels as a high-fidelity packet-level simulator, it is limited to modeling flow-level dynamics. It lacks support for capturing the complex task-level dependen-

⁴For the topologies with 1k, 2.5k, 5k, and 10k nodes, the total world sizes of all jobs are configured as 1024, 2048, 4096, and 8192, respectively. We ensure a node utilization rate of over 80% across the entire cluster.

Table 3: Full settings for all transport protocols.

Protocol	Parameters
Common settings	100 Gbps link speed, 1.5 KB MTU 40 cwnd, 500 pkt max queue size
NDP	80 pkt data/credit queue size
Bolt	1 MTU <i>CCThresh</i> , 1 MTU max <i>sm_token</i>
HPCC	$\eta = 95\%$, <i>max_stage</i> = 5
ExpressPass	60 pkt credit queue size 40 pkt Aeolus threshold 1 <i>winit</i> , 0.1 <i>target_loss</i>

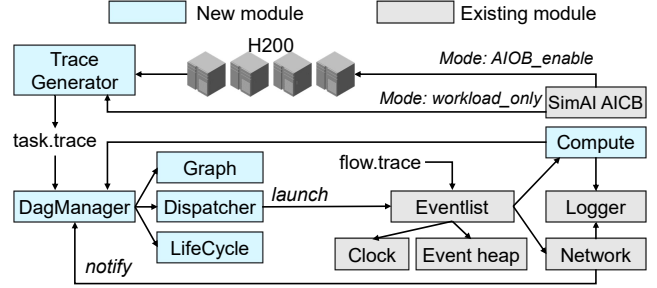


Figure 21: Extend htsim with a trace-driven DAG scheduler.

cies that arise in distributed AI training, where communication events can only be triggered after the completion of preceding computational kernels. To bridge the gap between pure network simulation and application-level AI dynamics, we extend *htsim* with a trace-driven DAG scheduler (Fig. 21).

To obtain task-level dependencies, we use SimAI to generate dependency-aware traces. SimAI is configured with the model parameters in Table 2 and produces profiling data in two modes. The *workload_only* mode captures communication semantics, including traffic volume, collective types, execution order, and group sizes, without executing kernels. The *AIOB_enable* mode samples realistic layer-wise computation latencies using PyTorch and CUDA on a physical GPU cluster. These outputs are fused by the Trace Generator into a *task.trace* that explicitly encodes computation–communication dependencies.

To process these traces and enforce dependencies, we extend *htsim* with a task-aware execution layer. The original discrete-event engine, including the Eventlist, Clock, Event heap, and the high-fidelity Network model, is preserved. A new DagManager parses the *task.trace* and dispatches tasks to the Network stack for communication or the virtual Compute module for computation.

Within the DagManager, the Graph module manages DAG dependencies, the Dispatcher module injects events into the Eventlist, and the LifeCycle handles task state transitions. The Compute module estimates kernel completion times based on the Clock. Upon completion of a compute event or network flow, a *notify* signal updates dependency states and releases successor tasks, ensuring strict causal ordering between computation and communication.

Algorithm 2 BUILDSTRUCTUREDBACKBONE

Require:

V : set of n nodes;
 k : target degree per node ($k = d - r$);
 $\mathcal{L}$: structured layout policy (w dimensions).

Ensure:

E_{local} : a k -regular undirected edge set.
1: $E_{\text{local}} \leftarrow \emptyset$, $\deg(u) \leftarrow 0, \forall u \in V$
 // Perform uniform sampling on a w -dimensional unit hypersphere to obtain node embeddings P_u , where $P = \frac{X}{\|X\|_2}, X \sim \mathcal{N}(0, I_w)$
2: $P \leftarrow \text{Place}(V, \mathcal{L})$
3: Compute similarity matrix $\mathbf{S} \in \mathbb{R}^{n \times n}$ where $S_{uv} = \langle P(u), P(v) \rangle$
 // Stage A: Local greedy filling via Top- T candidates
4: $\pi(u) \leftarrow$ Top- T neighbors of u sorted by $S_{u,\cdot}$ in descending order, $\forall u \in V$
5: **while** $\exists u \in V$ s.t. $\deg(u) < k$ **do**
6: Select $u \in \{x \in V \mid \deg(x) < k\}$
7: Find the first $v \in \pi(u)$ s.t. $\deg(v) < k$ and $(u, v) \notin E_{\text{local}}$
8: **if** such v exists **then**
9: $E_{\text{local}} \leftarrow E_{\text{local}} \cup \{(u, v)\}$
10: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
11: **else**
12: **break**
 // Stage B: Global augmentation via masked similarity
13: **if** $\exists u \in V$ s.t. $\deg(u) < k$ **then**
14: Initialize mask $\mathbf{M} \in \{0, 1\}^{n \times n}$ where $M_{uv} = 1 \iff u \neq v$
15: $M_{uv} \leftarrow 0, \forall (u, v) \in E_{\text{local}}$
16: $M_{u,\cdot} \leftarrow 0, M_{\cdot,u} \leftarrow 0, \forall u \in \{x \mid \deg(x) \geq k\}$
17: **while** $\exists u \in V$ s.t. $\deg(u) < k$ **do**
18: Select $u \in \{x \in V \mid \deg(x) < k\}$
19: $C \leftarrow \{x \in V \mid M_{ux} = 1\}$
20: **if** $C \neq \emptyset$ **then**
21: $v \leftarrow \arg \max_{x \in C} S_{ux}$
22: **else** //Random fall-back within unsatisfied nodes
23: $v \leftarrow$ any $x \in \{y \in V \setminus \{u\} \mid \deg(y) < k\}$
24: $E_{\text{local}} \leftarrow E_{\text{local}} \cup \{(u, v)\}$
25: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
26: $M_{uv} \leftarrow 0, M_{vu} \leftarrow 0$
27: **for** i in $\{u, v\}$ **do**
28: **if** $\deg(i) \geq k$ **then** $M_{i,\cdot} \leftarrow 0, M_{\cdot,i} \leftarrow 0$
29: **return** E_{local}

C LOONGX's Construction Details

For completeness, this appendix details the construction procedure for LOONGX, which specifies the topology instance used in our evaluation. The construction process is orchestrated by Alg. 1, which operates in two distinct phases: it first invokes BUILDSTRUCTUREDBACKBONE (Alg. 2) to establish a deterministic, locality-preserving base graph using the structured degree budget $d - r$ (line 2), and subsequently calls ADDSHORTCUTS (Alg. 3) to superimpose random edges with degree r for global reachability (line 3). The final topology is produced by merging these two edge sets (line 4) to yield a unified d -regular graph (line 5).

Alg. 2 constructs the edge set E_{local} . The procedure embeds nodes onto a w -dimensional unit hypersphere (line 2) and de-

Algorithm 3 ADDSHORTCUTS

Require:

V : set of n nodes;
 r : target shortcut degree per node;
 E_{avoid} : edge set that shortcuts must not overlap.

Ensure:

E_{short} : an r -regular expander edge set.
1: **if** $r = 0$ **then**
2: **return** $\emptyset$
3: $\mathcal{A} \leftarrow \{(u, v) \mid u, v \in V, u < v, (u, v) \notin E_{\text{avoid}}\}$
 // Candidate pool, try T_{max} times
4: **for** $t = 1 \dots T_{\text{max}}$ **do**
5: $E_{\text{short}} \leftarrow \emptyset$, $\deg(v) \leftarrow 0, \forall v \in V$
6: $\mathcal{A}_{\text{shuffled}} \leftarrow \text{Shuffle}(\mathcal{A})$
 // Greedy construction of an r -regular graph
7: **for** $(u, v) \in \mathcal{A}_{\text{shuffled}}$ **do**
8: **if** $\deg(u) < r$ **and** $\deg(v) < r$ **then**
9: $E_{\text{short}} \leftarrow E_{\text{short}} \cup \{(u, v)\}$
10: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
11: **if** $\forall x \in V, \deg(x) = r$ **then**
12: **break**
 // Check r -regularity before spectral test
13: **if** $\forall x \in V, \deg(x) = r$ **then**
14: $\mathbf{A} \leftarrow$ Adjacency matrix of (V, E_{short})
15: $\lambda_2 \leftarrow \max\{|\lambda| : \lambda \in \text{spec}(\mathbf{A}), \lambda \neq r\}$
16: **if** $\lambda_2 \leq 2\sqrt{r-1}$ **then**
17: **return** E_{short}
18: **return** $\emptyset$

rives a pairwise similarity matrix $\mathbf{S}$ (line 3). The construction then proceeds in two stages. Stage A (lines 4–13) prioritizes local density via a greedy filling strategy: nodes identify their Top- T nearest neighbors (line 4), and the algorithm iteratively connects unsatisfied nodes to their best available candidates, exiting early if local options are exhausted (line 12). Stage B (lines 15–34) acts as a global augmentation phase to enforce strict k -regularity. It initializes a mask to prevent duplicate connections (lines 16–18) and enters a loop to satisfy remaining degree constraints (line 19). For each node, it attempts to connect to the most similar valid peer (line 21), leveraging random selection only as a fallback (lines 24–26), while dynamically updating the mask to maintain graph simplicity (lines 26–29).

Alg. 3 generates the random shortcut layer. The process begins by populating a pool $\mathcal{A}$ with all potential edges that do not overlap with the existing structured backbone (line 2). To guarantee optimal spectral expansion, the construction is wrapped in an iterative search loop (line 3). In each iteration, the edge pool is shuffled, and edges are added greedily to build an r -regular subgraph (lines 5–11). Crucially, once regularity is achieved, the algorithm verifies the graph's expansion properties by computing the second largest eigenvalue λ_2 of its adjacency matrix (lines 14–15). The candidate graph E_{short} is accepted only if it satisfies or strictly approaches the bound $\lambda_2 \leq 2\sqrt{r-1}$ (lines 16–17). Together, they yield a d -regular LOONGX topology with local density and global reachability.

D Proof of the Clustering Coefficient Decay with Dimension

This section proves that, in LOONGX, the clustering coefficient decreases monotonically as the dimension w increases.

Theorem 3.1 (Clustering Coefficient Decay with Dimension). *For a LOONGX topology with a fixed local degree $k = d - r$ and a total number of nodes $N \gg k$, the local clustering coefficient $C(w)$ decreases as the number of logical dimensions w increases.*

Proof. The clustering coefficient $C(w)$ is formally defined as the probability that two randomly selected neighbors of a node u are connected. Let $\mathcal{N}(u)$ denote the neighbor set of u . For any two neighbors $v_1, v_2 \in \mathcal{N}(u)$, the coefficient is given by the conditional probability:

$$C(w) = \Pr(v_1 \sim v_2 \mid v_1, v_2 \in \mathcal{N}(u)). \quad (1)$$

In the geometric construction of LOONGX, where connectivity is determined by proximity within a unit ball $B(\cdot)$, this probability is equivalent to the expected intersection volume of the balls $B(u)$ and $B(v_1)$, normalized by the volume of a single ball V_m .

Let $m = w - 1$ be the manifold dimension. The distance $x = \|v_1 - u\|$ follows the probability density function $f(x) = mx^{m-1}$ for $x \in [0, 1]$. For a given distance x , the normalized intersection volume $I_m(x)/V_m$ is:

$$\frac{I_m(x)}{V_m} = \frac{2V_{m-1}}{V_m} \int_{x/2}^1 (1-t^2)^{\frac{m-1}{2}} dt. \quad (2)$$

The total clustering coefficient is the expectation over x :

$$C(w) = \int_0^1 mx^{m-1} \left(\frac{2V_{m-1}}{V_m} \int_{x/2}^1 (1-t^2)^{\frac{m-1}{2}} dt \right) dx. \quad (3)$$

To demonstrate the monotonic decrease with respect to w , we observe that for any $t \in (1/2, 1)$, the integrand term $(1-t^2)^{\frac{m-1}{2}}$ decreases exponentially as m and thus w increases. Geometrically, as the dimensionality increases, the volume of the unit ball concentrates near the boundary, while the overlap region between $B(u)$ and $B(v_1)$ occupies a shrinking fraction of the total volume.

Asymptotically, the integral is dominated by the behavior near the lower limit $t = 1/2$. Therefore:

$$C(w) \lesssim \left(\sqrt{1 - (1/2)^2} \right)^m = \left(\frac{3}{4} \right)^{\frac{w-1}{2}}. \quad (4)$$

The function $\left(\frac{3}{4} \right)^{\frac{w-1}{2}}$ is strictly decreasing in w . This proves that increasing the dimension w of LOONGX reduces the clustering coefficient $C(w)$. $\square$

Algorithm 4 INCREMENTALEXPANSION

Require:

$LX(n, d, r, w) = (V, E)$: current graph separating E_{base} and E_{rand} ;
 m : number of new nodes to add;
 d : number of uplinks per node;
 r : number of random edges per node.

Ensure:

$LX(n, d, r, w)'$: updated graph with $|V| + m$ nodes preserving LoongX properties.

```

1: for  $i \leftarrow 1$  to  $m$  do
2:   Create new node  $c$ ,  $V \leftarrow V \cup \{c\}$ 
   // Phase 1: Structured Backbone Extension (Local Full-Mesh)
3:   Select insertion position  $pos$  based on logical layout  $\mathcal{L}$ 
4:   Identify local neighbor group  $\mathcal{G}$  of size  $(d - r)$  around  $pos$ 
5:    $E_{\text{base}} \leftarrow \text{ReMeshToSubgraph}(\mathcal{G} \cup \{c\})$ 
   // Phase 2: Random Shortcut Extension (Min-Perturbation)
6:   if valid matching of size  $r/2$  exists in  $E_{\text{rand}}$  then
7:      $\mathcal{M} \leftarrow$  Randomly select  $r/2$  disjoint edges from  $E_{\text{rand}}$ 
     // Edges  $\{(u_1, v_1), \dots, (u_{r/2}, v_{r/2})\}$  do not share vertices
8:     for  $(u, v) \in \mathcal{M}$  do
9:        $E_{\text{rand}} \leftarrow E_{\text{rand}} \setminus \{(u, v)\}$  // Remove existing edge
10:       $E_{\text{rand}} \leftarrow E_{\text{rand}} \cup \{(u, c), (v, c)\}$  // Fold edges to node  $c$ 
11:  $E \leftarrow E_{\text{base}} \cup E_{\text{rand}}$ 
12: return  $LX(n, d, r, w)' = (V, E)$ 

```

E Incremental Expansion Procedure

To facilitate flexible cluster growth, LOONGX implements a low-overhead expansion mechanism that allows new nodes to be integrated online with minimal topological disturbance. Alg. 4 details the incremental expansion procedure. Iterating through each new node c (lines 1–2), the process first updates the structured backbone to preserve locality (phase 1): it determines the logical insertion position and identifies the corresponding local neighbor group (lines 3–4), then integrates c by locally remeshing these nodes into a full-mesh subgraph structure (line 5). Subsequently, to establish global connectivity with minimal network disturbance (phase 2), the algorithm employs an edge-folding strategy: it identifies $r/2$ disjoint edges from the existing random graph (line 7) and redirects their endpoints to connect with c (lines 9–10), thereby satisfying the random degree requirement r while maintaining the topology’s overall expansion properties.

F Topological Hop Distribution

Beyond average path length, collective-heavy AI workloads are sensitive to latency skew across node pairs, where a long-hop tail can stall synchronized collectives such as all-reduce. We therefore report the distribution of shortest-path hop counts at 2.5k and 10k nodes in Fig. 22.

LOONGX keeps path lengths tightly bounded as the cluster scales. At 2.5k nodes, all LOONGX paths complete within five hops, with most concentrated at four and five hops. At 10k nodes, LOONGX still avoids long-hop tails: 96.9% of paths are within five hops. The resulting distribution closely tracks Expander, showing that LOONGX preserves near-random-

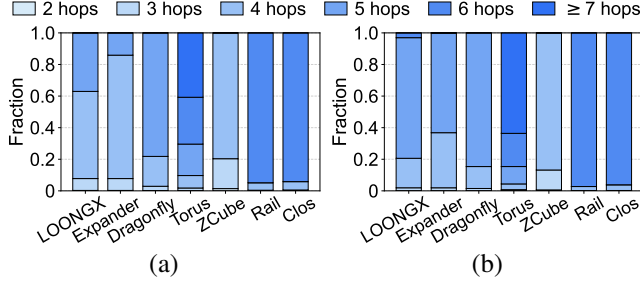


Figure 22: Hop count distribution at different cluster scales: (a) 2.5k and (b) 10k nodes.

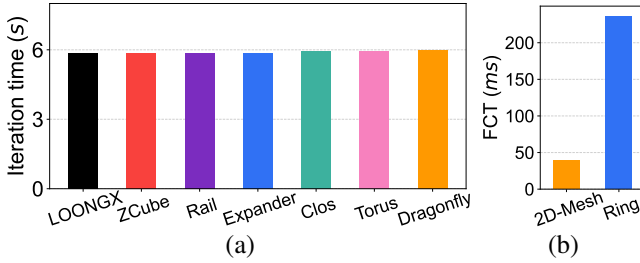


Figure 23: (a) Iteration time for various topologies. (b) FCT of all-reduce on LOONGX under 2D-Mesh and Ring schemes.

graph path lengths with only limited random shortcuts.

Other baselines exhibit more pronounced structural effects. Torus shows a long-hop tail as scale increases, with 63.6% of paths requiring at least seven hops at 10k nodes. Clos and Rail-optimized designs concentrate most paths at six hops due to fixed hierarchical traversal, while Dragonfly shifts many paths to five hops once communication crosses groups. Although LOONGX is not symmetric, its hop-count distribution stays close to Expander and tightly bounded, avoiding the long-hop tails of Torus, Clos, and Rail-optimized designs that would otherwise widen the latency skew seen by collectives.

G Dense Model Performance

We conduct GPT3 training experiments on 1k nodes with a parallelism configuration of TP=8, PP=8, and DP=16. Fig. 23a illustrates that the iteration time remains remarkably consistent across different topologies. This insensitivity arises because TP communication is highly localized within a single server, thereby decoupling it from the core network, while PP traffic remains minimal. Furthermore, although the network structure affects the efficiency of DP all-reduce, the pipelining mechanism effectively masks this overhead; consequently, DP traffic constitutes a negligible fraction of the total iteration time for dense models.

Fig. 23b compares the FCT of DP all-reduce on the LOONGX topology under the Ring and 2D-Mesh schemes, where clear performance differences are observed. The results indicate that the 2D-Mesh algorithm dramatically reduces FCT compared to the Ring approach, demonstrating that the structural characteristics of LOONGX are intrinsically well-suited for mesh-based collective communication.

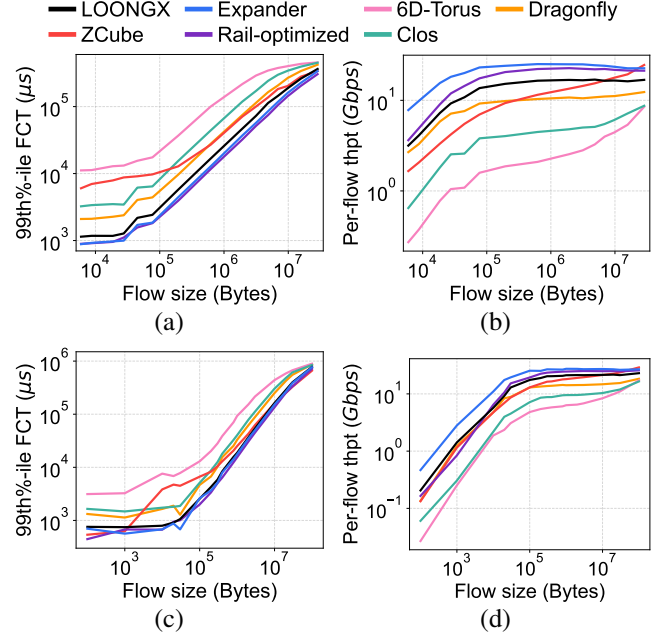


Figure 24: Performance under non-AI traffic: (a) 99%-ile FCT and (b) per-flow throughput under Websearch workload; (c) 99%-ile FCT and (d) per-flow throughput under Meta trace.

H Non-AI Traffic Performance

For non-AI traffic, we evaluate Websearch [4] and Meta traces [17] at 2.5k nodes. We set the core network load to 50% as a heavy-load regime, exceeding typical production data centers where median link utilization is 10%–20% [48] and over 80% of links operate below 10% utilization [62].

FCT. Fig. 24a and Fig. 24c compare the 99%-ile FCT across two traces. Expander achieves the lowest tail latency, followed by Rail-optimized and LOONGX, while 6D-Torus performs the worst. Under the Websearch trace, the 99%-ile FCT of LOONGX is only 18% higher than Expander, whereas Torus is $5\times$ worse. This trend follows their diameter and APL: Expander minimizes both, Torus has the largest diameter, and LOONGX incurs a higher APL. Despite this gap, LOONGX remains competitive for non-AI traffic due to its lower cabling complexity, and we show that it outperforms Expander for AI workloads with more localized communication.

Per-flow throughput. Fig. 24b and Fig. 24d report the per-flow throughput [41]. Per-flow throughput is computed as the flow size divided by its FCT, which reflects the effective bandwidth obtained by each flow. Expander achieves the highest per-flow throughput, followed by Rail-optimized, LOONGX, ZCube, and Dragonfly, with 6D-Torus performing the worst. This ordering is consistent with the FCT results. Under the Meta trace, Expander achieves 25.2% higher per-flow throughput than LOONGX, 26.4% higher than ZCube, and 78.9% higher than Dragonfly. Overall, the per-flow throughput trends closely follow the FCT performance and can be explained by the same underlying topological properties.

Table 4: Topology configuration and basic properties at different scales.

Topo. scale (#nodes)	Topology	Configuration							Property (node-to-node)			
		Switch	Uplink/ Downlink	Radix	TX/RX	NIC	Electrical switch port	Cost (M. \$)	Diameter	APL	APL _{fail}	Clustering coefficient
1k	LOONGX (128,24,6,1)	128	3:1	32	5120	1024	4096	1.948	5	3.84	3.90	0.446
	Expander	128	3:1	32	5120	1024	4096	1.948	5	3.80	3.83	0.177
	Dragonfly	264	3:1	16	5248	1024	4224	1.984	5	4.68	4.86	0.382
	6D-Torus	256	3:1	16	5120	1024	4096	1.948	10	6.00	6.02	0.000
	ZCube	(100,200)	1:1	(30,20)	6000	1000	5000	2.188	4	3.69	3.73	–
	Rail-optimized	320	1:1	16	6144	1024	5120	2.241	6	5.86	5.86	–
	Clos	215	1:2	15	4275	1050	3225	1.718	6	5.85	5.85	–
2.5k	LOONGX (320,24,6,1)	320	3:1	32	12800	2560	10240	4.869	5	4.29	4.37	0.412
	Expander	320	3:1	32	12800	2560	10240	4.869	5	4.06	4.13	0.071
	Dragonfly	510	3:1	20	12750	2550	10200	4.850	5	4.75	4.95	0.396
	6D-Torus	648	3:1	16	12960	2592	10368	4.930	9	6.17	6.17	0.073
	ZCube	(196,392)	1:1	(42,28)	16464	2744	13720	6.004	4	3.78	3.82	–
	Rail-optimized	605	1:1	22	15246	2541	12705	5.560	6	5.90	5.90	–
	Clos	376	1:2	21	10416	2520	7896	4.168	6	5.88	5.88	–
5k	LOONGX (640,24,6,1)	640	3:1	32	25600	5120	20480	9.738	6	4.58	4.64	0.400
	Expander	640	3:1	32	25600	5120	20480	9.738	5	4.35	4.43	0.035
	Dragonfly	876	3:1	24	26280	5256	21024	9.997	5	4.80	5.00	0.404
	6D-Torus	1296	3:1	16	25920	5184	20736	9.860	9	6.33	6.34	0.076
	ZCube	(289,578)	1:1	(51,34)	29478	4913	24565	13.789	4	3.82	3.86	–
	Rail-optimized	980	1:1	28	30576	5096	25480	10.750	6	5.92	5.92	–
	Clos	711	1:2	27	24345	5148	19197	9.393	6	5.90	5.90	–
10k	LOONGX (1280,24,6,1)	1280	3:1	32	51200	10240	40960	19.476	6	4.80	4.88	0.397
	Expander	1280	3:1	32	51200	10240	40960	19.476	6	4.61	4.67	0.017
	Dragonfly	1386	3:1	28	48510	9702	38808	18.453	5	4.83	5.05	0.411
	9D-Torus	1728	3:1	24	51840	10368	41472	19.720	11	7.00	7.00	0.046
	ZCube	(484,968)	1:1	(66,44)	63888	10648	53240	23.298	4	3.86	3.90	–
	Rail-optimized	1532	1:1	35	63070	10115	52955	22.812	6	5.94	5.94	–
	Clos	959	1:2	33	41855	10208	31647	16.789	6	5.93	5.93	–

Table 5: Testbed topology configuration.

Topo. scale (#nodes)	Topology	Switch	Uplink/Downlink	Radix	TX/RX	NIC	Electrical switch port	Cost (M. \$)
256	LOONGX (64,6,2,1)	64	2:1	12	1024	256	768	0.411
	Expander	64	2:1	12	1024	256	768	0.411
	Dragonfly	64	2:1	12	1024	256	768	0.411
	4D-Torus	64	2:1	12	1024	256	768	0.411
	Clos	(48,32)	1:2	(8,12)	1024	256	768	0.411
	ZCube	32	1:1	32	1024	256	768	0.411

Table 6: Cost of network components.

Link bandwidth	TX/RX (\$)	NIC (\$)	Electrical switch port (\$)
100Gbps	99	650	187
200Gbps	239	1079	374
400Gbps	659	1499	1090
800Gbps	1399	2248	1400